Recipe Rating Predictions Based on MEAT Inclusion

Name(s): Evelyn Zhang, Haowen Zhang

Website Link: https://evelynzhang5.github.io/recipe_rating/

In [1]:
```
import pandas as pd
import numpy as np
from pathlib import Path

import plotly.express as px
pd.options.plotting.backend = 'plotly'
from dsc80_utils import *
```

In [2]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV, KFold
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, FunctionTra
from sklearn.decomposition import PCA
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.preprocessing import MultiLabelBinarizer


from sklearn.pipeline import Pipeline as SkPipeline
from sklearn.preprocessing import FunctionTransformer, StandardScaler, OneHo
from sklearn.decomposition import PCA
from sklearn.compose import ColumnTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, KFold, GridSearchCV
```

Step 1: Introduction

Loading [MathJax]/extensions/Safe.js

Over the past few years there has been a major change in dietary preferences and more people are paying attention to how the composition of recipes influences consumer perceptions and ratings. Specifically, the inclusion of meat in recipes has become a focal point, as trends toward plant-based eating has been popularized with our increasing health-conscious and environmental awareness.

Therefore, we are interested in investigating what kind of reciptes tend to have a higher ratings and hence predict the ratings of reciptes.

Step 2: Data Cleaning and Exploratory Data Analysis

Data Cleaning

We used the following steps to clean our data:

1. Left merge `recipe` and `interactions` dataset.

2. Check the data type for resulting columns

3. Fill all ratings of **0** in `rating` with **np.nan** and convert th rest to integers.

    - All the ratings have scale from **1** to **5** with **1** meaning the lowest rating **5** means the highest rating; therefore, a rating of **0** suggests that the rating for that speicifc recipe is missing. Thus, to avoid bias in the `ratings` and false results during our permutation test, we filled the value **0** with **np.nan**. Since we plan to build classifier, we changed the type to interger.
4. Check the **null** values across columns. Drop one row with `name` of the recipe being **nan**.

5. Add column `average_rating` containing average ratings per recipe.

    - Since a recipe can have numerous ratings from different users while we are merging the data, we take the **mean** of all the ratings, which will give us a more comprehensive understanding of all the ratings for the given recipe.
6. Split and convert the `nutritions`, `ingredients`, `tags` column from `str` to `list`.

    - Since we will be doing multiple extractions and look up of items in each column for future analysis, we thought of converting these columns which are **objects** types, acting like strings, to **list** type. Speicifically, we

Loading [MathJax]/extensions/Safe.js

applied a lambda function to perform simple strip() and split() then converted the columns to **list** of strings or floats depending on the content. It will allow us to conduct numerical calculations, item lookup, and so forth on the columns.

7. Split values in the `nutrition` column to individual columns of floats

   - After converting the column to **list**, we then seperate each into a column with thecorresponding nutrition information as **float**. This could help us look up for speicifc nutrition information more quickly.

8. Convert `submitted` and `date` to datetime.

   - Sinnce these two columns are both stored as **objects**, for our convenience, we decidded to convert them into datetime to allow us conduct analysis on selected period of time.

9. Add `contains_meat` to the dataframe

   - The `contains_meat` is a boolean column checking if the tags of recipes contain `meat`. This step separates the recipes into two groups, with recipes contain `meat` and those without. This provides us a convenient way to compare ratings of recipes of two distributitons: recipes with and without `meat`.

In [3]:
```
recipes= pd.read_csv('RAW_recipes.csv')
interactions = pd.read_csv('RAW_interactions.csv')
recipes_df = recipes.drop(columns=['Unnamed: 0'], errors='ignore')
# Left merge the recipes and interactions datasets on 'id' (recipe_id in int
merged_df = recipes_df.merge(interactions, left_on='id', right_on='recipe_ic
# Convert rating column to numeric explicitly
merged_df['rating'] = pd.to_numeric(merged_df['rating'], errors='coerce')
# Replace all ratings of 0 with NaN
merged_df.loc[merged_df['rating'] == 0, 'rating'] = np.nan
# Calculate the average rating per recipe
average_ratings = merged_df.groupby('id')['rating'].mean()
# Merge the average ratings back into the recipes dataset
recipes_df = merged_df.merge(average_ratings, on='id', how='left').rename(cc
```

In [4]:
```
cleaned_df = recipes_df
cleaned_df['nutrition'] = cleaned_df['nutrition'].apply(lambda x: (x.strip("
cleaned_df['ingredients'] = cleaned_df['ingredients'].apply(lambda x: (x.str
cleaned_df['date']= pd.to_datetime(cleaned_df['date'])
cleaned_df['submitted']=pd.to_datetime(cleaned_df['submitted'])
cleaned_df['tags'] = cleaned_df['tags'].apply(lambda x: [item.strip(" '") fo
cleaned_df['contains_meat']=cleaned_df['tags'].apply(lambda x: 'meat' in x)
# Expand the list into individual columns. Adjust the column names as desire
cleaned_df[['calories', 'total_fat', 'sugar', 'sodium', 'protein', 'saturate
    pd.DataFrame(cleaned_df['nutrition'].tolist(), index=cleaned_df.index)
# Convert the new columns to float type
cols = ['calories', 'total_fat', 'sugar', 'sodium', 'protein', 'saturated_fa
cleaned_df[cols] = cleaned_df[cols].astype(float)
cleaned_df['rating']= cleaned_df['rating'].apply(lambda x: int(x) if pd.notr
```

Loading [MathJax]/extensions/Safe.js

```
cleaned_df= cleaned_df.set_index('name')
cleaned_df
```

Out[4]:

name	id	minutes	contributor_id	submitted	...	sodium	protein
1 brownies in the world best ever	333281	40	985201	2008-10-27	...	3.0	3.0
1 in canada chocolate chip cookies	453467	45	1848091	2011-04-11	...	22.0	13.0
412 broccoli casserole	306168	40	50969	2008-05-30	...	32.0	22.0
...	...	...	...	...	...	...	...
cookies by design sugar shortbread cookies	298509	20	506822	2008-04-15	...	4.0	4.0
cookies by design sugar shortbread cookies	298509	20	506822	2008-04-15	...	4.0	4.0
cookies by design sugar shortbread cookies	298509	20	506822	2008-04-15	...	4.0	4.0

234429 rows × 25 columns

Univarate Analysis

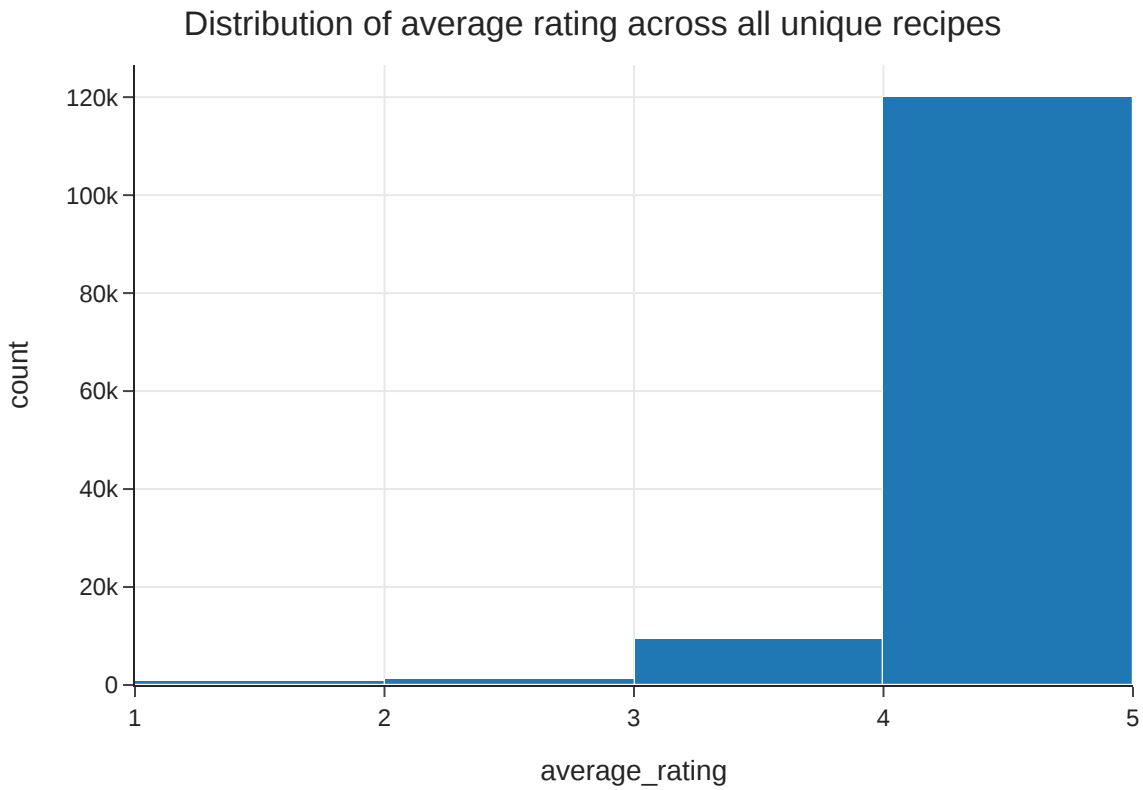
Since we would like to classify the ratings in the end, we would like to understand the distribution of our `average_ratings`. As we can observe in the graph, the recipe ratings are more concentrated towards 4 or 5, meaning there is a clear left skew. We suspect that the `average_ratings` might be biased and people tend to give ratings for review especially when they particular like the receipe.

In [5]:

```
import plotly.io as pio
#pio.renderers.default = "iframe"
pio.renderers.default = "notebook"
```

Loading [MathJax]/extensions/Safe.js

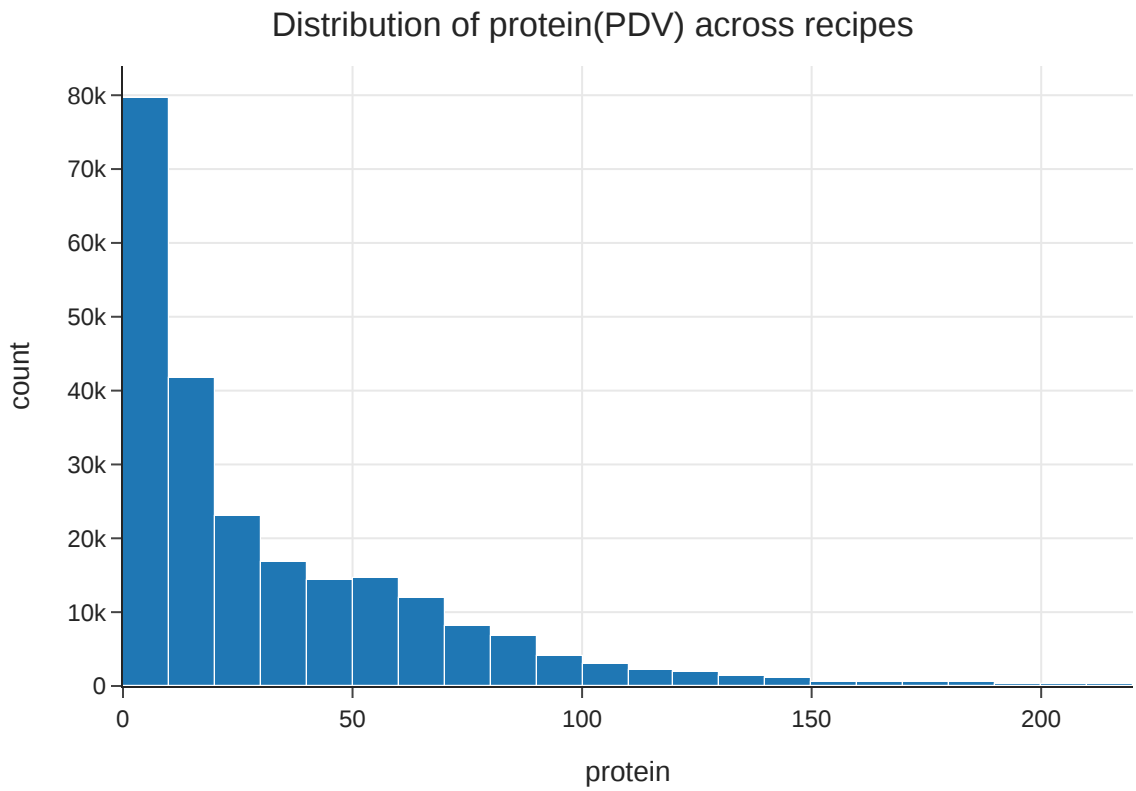
```
# or
fig = px.histogram(cleaned_df, x='average_rating', nbins=5, title='Distribut
fig.update_traces(xbins=dict(start=1, end=5, size=1))
fig.show()
fig.write_html('rating-distribution.html', include_plotlyjs='cdn')
```



We would like to analyze the relationships between the ratings and whether the recipes contains meat. The protein content is associated with whether the recipe contains meat. Se we make a histogram to learn the distribution of `protein`.

In [6]:
```
import plotly.io as pio
#pio.renderers.default = "iframe"
pio.renderers.default = "notebook"
# or

fig = px.histogram(cleaned_df, x='protein', nbins=5, title='Distribution of
fig.update_traces(xbins=dict(start=0, end=220, size=10))
fig.show()
fig.write_html('protein-distribution.html', include_plotlyjs='cdn')
```

Loading [MathJax]/extensions/Safe.js



From the graph above, we learn that the distribution of protein skews to the right. Most of the dishes have protein content between 0 and 25. This might indicate that recipe containing meat may lead to lower ratings, we will test it later.

Bivariate Analysis

To investigate our most interested question,**will the meat inclusion(with meat tag) affect recipes' ratings**, we used a KDE plot to show the distribution of average ratings for recipes with and without the meat tag. We can see that both curves, with and without meat tags, have a major peak around a certain rating (around 4-5). However, in general, the close alignment of the two KDE curves implies that the presence or absence of meat **does not** significantly shift the overall rating distribution.

In [7]:
```
pip install mpld3
```

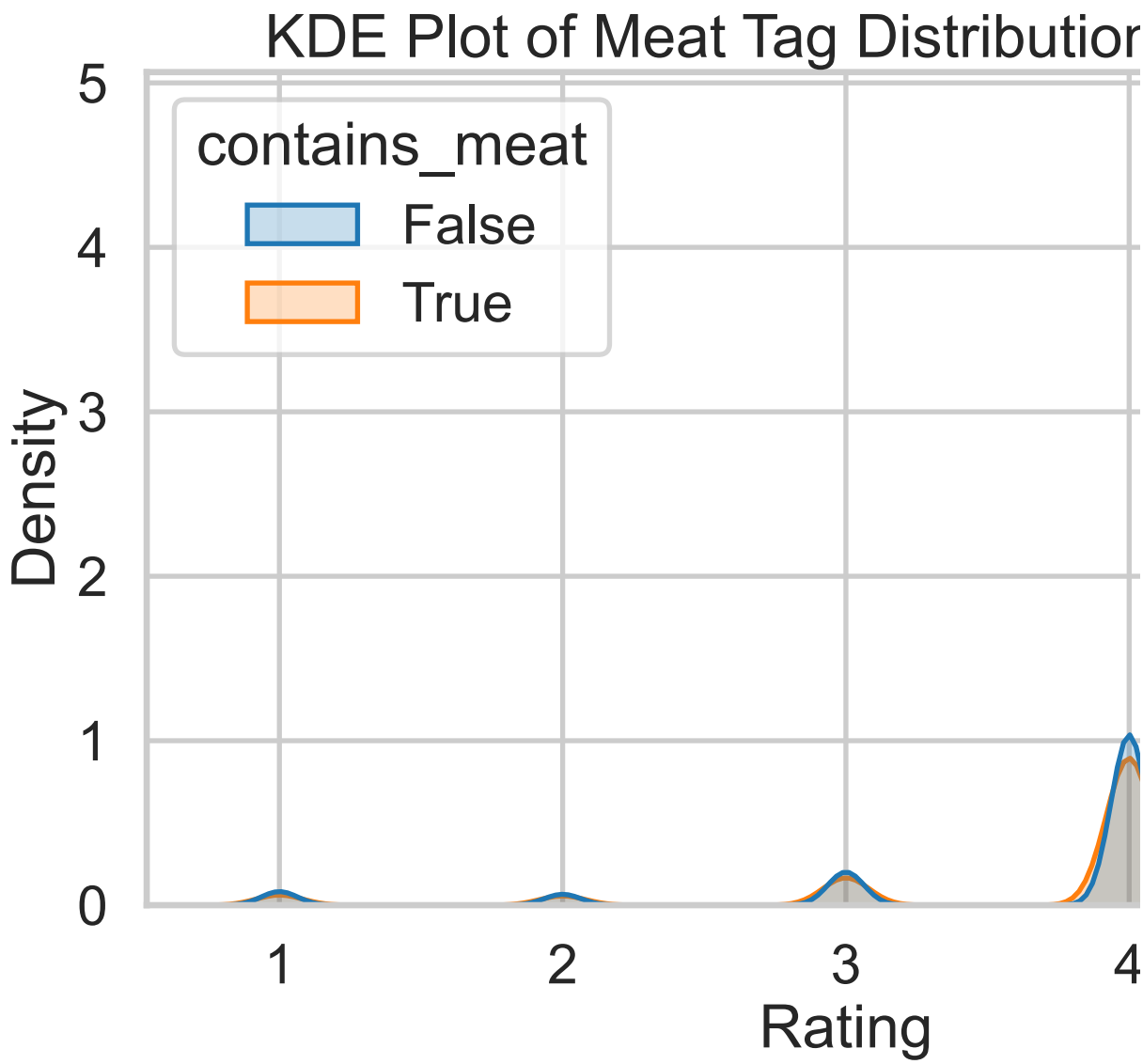
Loading [MathJax]/extensions/Safe.js

```
Requirement already satisfied: mpld3 in /opt/anaconda3/envs/dsc80/lib/python
3.12/site-packages (0.5.10)
Requirement already satisfied: jinja2 in /opt/anaconda3/envs/dsc80/lib/pytho
n3.12/site-packages (from mpld3) (3.1.5)
Requirement already satisfied: matplotlib in /opt/anaconda3/envs/dsc80/lib/p
ython3.12/site-packages (from mpld3) (3.9.2)
Requirement already satisfied: MarkupSafe>=2.0 in /opt/anaconda3/envs/dsc80/
lib/python3.12/site-packages (from jinja2->mpld3) (3.0.2)
Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/envs/dsc8
0/lib/python3.12/site-packages (from matplotlib->mpld3) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /opt/anaconda3/envs/dsc80/li
b/python3.12/site-packages (from matplotlib->mpld3) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/envs/dsc8
0/lib/python3.12/site-packages (from matplotlib->mpld3) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/envs/dsc8
0/lib/python3.12/site-packages (from matplotlib->mpld3) (1.4.8)
Requirement already satisfied: numpy>=1.23 in /opt/anaconda3/envs/dsc80/lib/
python3.12/site-packages (from matplotlib->mpld3) (2.1.1)
Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/envs/dsc80/
lib/python3.12/site-packages (from matplotlib->mpld3) (24.2)
Requirement already satisfied: pillow>=8 in /opt/anaconda3/envs/dsc80/lib/py
thon3.12/site-packages (from matplotlib->mpld3) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/envs/dsc8
0/lib/python3.12/site-packages (from matplotlib->mpld3) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda3/envs/d
sc80/lib/python3.12/site-packages (from matplotlib->mpld3) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/envs/dsc80/lib/pyt
hon3.12/site-packages (from python-dateutil>=2.7->matplotlib->mpld3) (1.17.
0)
Note: you may need to restart the kernel to use updated packages.
```

In [8]:

```python
import matplotlib.pyplot as plt
import seaborn as sns

# Create a figure and axis object
fig, ax = plt.subplots(figsize=(10, 6))

# Create the KDE plot with a separate density for recipes with and without m
sns.kdeplot(
    data=cleaned_df.reset_index(),
    x="rating",
    hue="contains_meat",
    fill=True,           # Fills the area under each KDE curve
    common_norm=False,   # Normalize densities independently
    ax=ax
)
# Set title and labels
ax.set_title("KDE Plot of Meat Tag Distribution by Rating")
ax.set_xlabel("Rating")
ax.set_ylabel("Density")

plt.show()

import mpld3
mpld3.save_html(fig, "KDE.html")
```

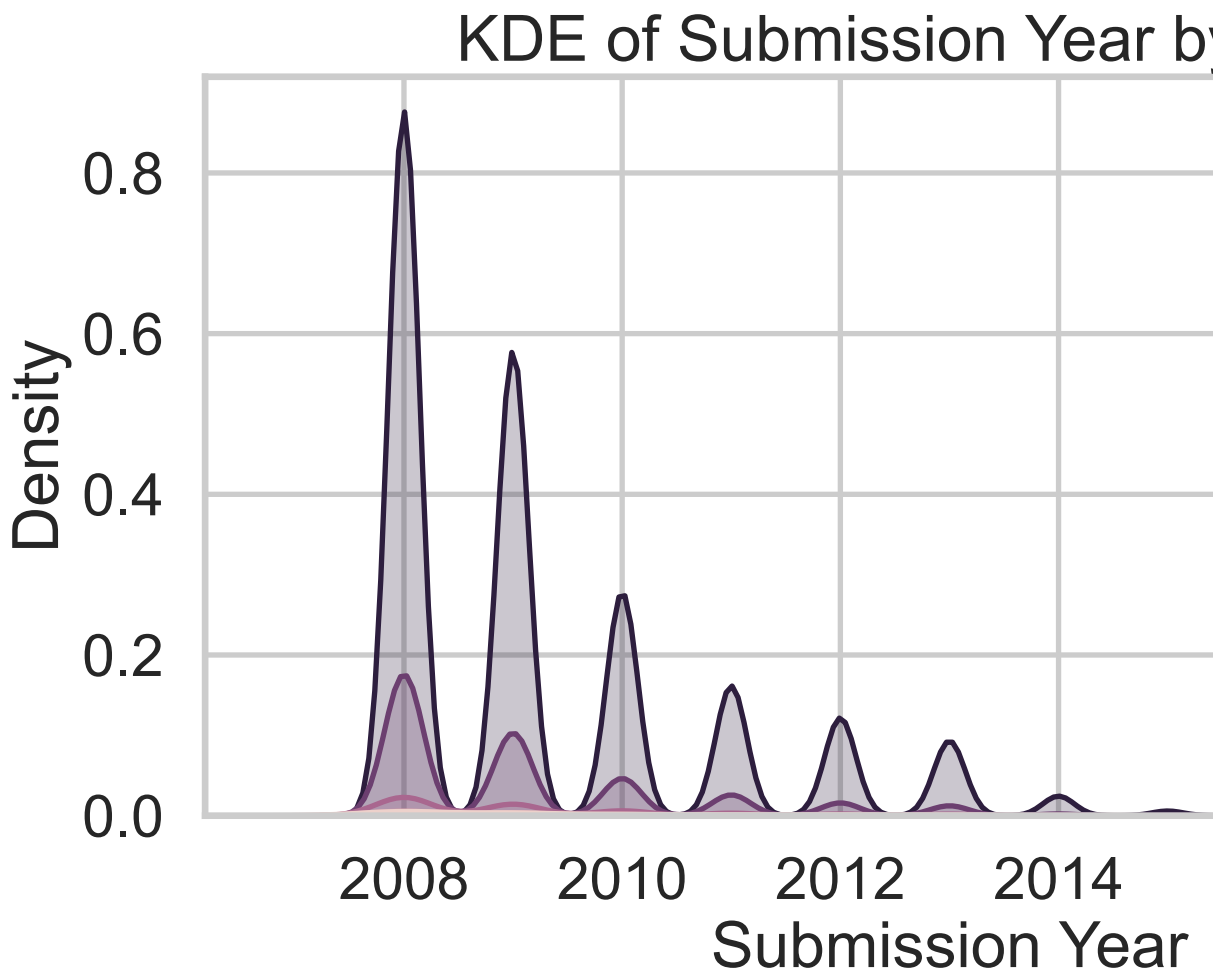
Loading [MathJax]/extensions/Safe.js



Similarly, in this KDE graph between **submitted_year** and **rating**, we see that more ratings and higher ratings are given in earlier years. In other words, recipes submitted in recent years has a lower rating.

In [9]:
```
import seaborn as sns
import matplotlib.pyplot as plt
df = cleaned_df.copy().dropna()
df['submitted_year'] = df['submitted'].dt.year

sns.kdeplot(data=df, x="submitted_year", hue="rating", fill=True)
plt.xlabel("Submission Year")
plt.ylabel("Density")
plt.title("KDE of Submission Year by Rating")
plt.show()
```

Loading [MathJax]/extensions/Safe.js



Interesting Aggregates

For our aggregating analysis, we would like to use pivot table to compare and contrast the difference in `rating`, `protein`, `minutes` and `total_fat` between two distributions in `contains_meat` group (with and without meat tag) Interestingly, we can see that recipes with meat deliver higher protein but also come with higher fat content and longer preparation times.

In [10]:
```
import pandas as pd

# Create a pivot table that computes the mean values for average_rating and
pivot_table = cleaned_df.pivot_table(
    index='contains_meat',
    values=['rating', 'total_fat','protein','minutes'],
    aggfunc='mean'
)

print(pivot_table)
```

```
               minutes  protein  rating  total_fat
contains_meat
False           103.20    22.84    4.68      28.05
True            118.35    66.42    4.67      44.39
```

Loading [MathJax]/extensions/Safe.js

Another analysis we performed is to observe the trend of distributions of nutrition information based on `rating`. Recipes **with meat** tend to have **higher** calories, protein, total fat, and saturated fat, while non-meat recipes show higher carbohydrate` and sugar values. By observing these trends, we can identify which features may have predictive/determining power for our classifier.

In [11]:
```
pivot_table = cleaned_df.pivot_table(
    index='rating',
    values=['minutes', 'calories', 'total_fat', 'sugar', 'protein', 'saturat
    aggfunc='mean'
)


print(pivot_table.to_markdown())
```

```
|   rating |   calories |   carbohydrates |   minutes |   n_ingredients |
protein |   saturated_fat |   sugar |   total_fat |
|---------:|-----------:|----------------:|----------:|----------------:|---
-------:|----------------:|--------:|------------:|
|        1 |    486.595 |         16.3962 |   99.6725 |         8.91289 |
34.0589 |         46.6791 | 88.0094 |     37.0564 |
|        2 |    446.598 |         15.0405 |   98.0215 |         9.22973 |
34.307  |         42.8822 | 75.1664 |     32.7669 |
|        3 |    425.791 |         13.6813 |   87.4976 |         9.20092 |
34.8582 |         40.0881 | 65.552  |     31.6405 |
|        4 |    405.047 |         12.8306 |   91.585  |         9.10076 |
34.0466 |         36.4327 | 56.7858 |     29.9404 |
|        5 |    415.213 |         13.038  |  106.924  |         9.04675 |
32.6446 |         39.2268 | 63.0816 |     31.7924 |
```

Overall, the **nutritional** metrics (calories, carbohydrates, protein, saturated fat, sugar, and total fat) show a fairly clear negative trend with increasing rating—lower values tend to correspond to higher ratings (at least from ratings 1 to 4). The decrease in calories, carbs, and fats from ratings 1 to 4 might reflect a preference for recipes that are perceived as healthier or less indulgent. However, the slight reversal in rating 5 (longer minutes, higher `fat` / `sugar`) could suggest that the most highly rated recipes balance healthiness with complexity or flavor richness that requires more preparation time and slightly richer ingredients.

Most metrics **decrease** as the rating increases, suggesting that higher ratings (in this range) might be associated with healthier or less energy-dense options. There are some small increase sin several values at rating 5 indicates that while there's an overall downward trend, the highest rating category might not follow the pattern as closely as ratings 1 to 4, so there might be some outliers existing.

Step 3: Assessment of Missingness

Loading [MathJax]/extensions/Safe.js

The three columns in the cleaned dataframe that have non-trivial number missing entries are `rating` (**15036**), `review` (**2777**), and `description` (**136**).

NMAR Analysis

In these three columns, we think that **none** is **NMAR**.

In fact, we believe that all three columns all could be **MAR** due to the fact that they tend to correlate with each other. For instance, people who like the recipe will likely to give `rating` and also leave some positive feedback and `review` or the `description` of the dish itself to express his or her enjoyment. Also, the activeness/number of the `review`, `description`, and the `rating` could be also dependent on the `contributor_id` and the `name` of the recipe. One user could be really proactive in delievering messages or leave comments. One recipe may be really special/seasonal/memroable or for is designed for a specific holiday, leading to more views and attention.

Specifically, we believe that `rating` is strongly likely to be influneced/correlated with other columns.

Missingness Dependency

First, we would like to investigate whether the number of ingredients has an impact on the missingness of `rating`.

`n_ingredients` and `rating`'s missing dependecy

Null Hypothesis: The missingness of ratings does not depend on the number of ingredients(`n_ingredients`) in the recipe.

Alternative Hypothesis: The missingness of ratings does depend on the number of ingredients(`n_ingredients`) in the recipe.

Test Statistic: the absolute difference in mean `n_ingredients` between missing and non-missing rating groups.

Significance level: 0.01

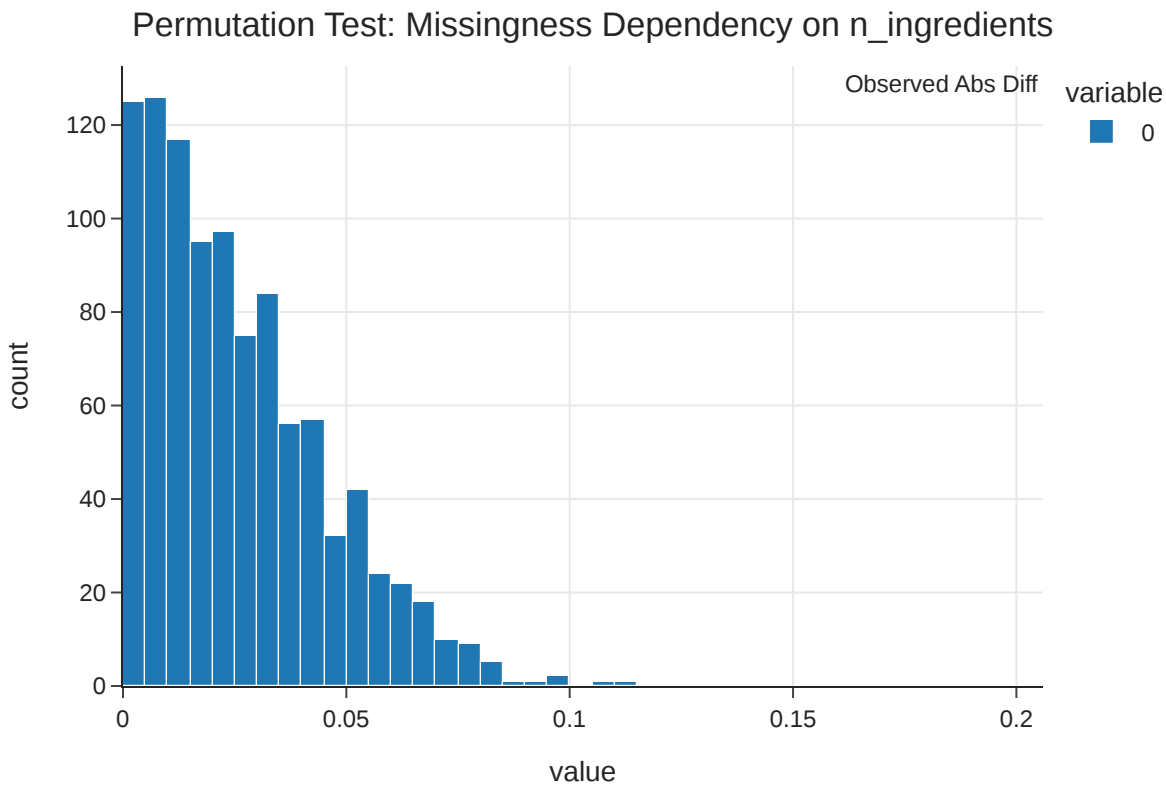
Here is our **result**:

In [12]:
```
import numpy as np
import pandas as pd
import plotly.express as px
from scipy.stats import norm

df = cleaned_df.copy()
df = df.reset_index()[['name', 'n_ingredients', 'rating']]
```

Loading [MathJax]/extensions/Safe.js

```
# Create missingness indicator for rating
df["rating_missing"] = df["rating"].isna()

# Compute observed absolute difference in mean n_ingredients between missing
observed_diff_missing = abs(df.groupby("rating_missing")["n_ingredients"].me

# Permutation test setup
num_permutations = 1000
perm_diffs_missing = []

for _ in range(num_permutations):
    permuted = df.copy()
    permuted["rating_missing"] = np.random.permutation(permuted["rating_miss
    perm_diff = abs(permuted.groupby("rating_missing")["n_ingredients"].mear
    perm_diffs_missing.append(perm_diff)

# Compute p-value (two-sided)
p_value_missing = np.mean(np.array(perm_diffs_missing) >= observed_diff_miss

# Plot permutation results
fig_missing = px.histogram(perm_diffs_missing, nbins=30,
                           title="Permutation Test: Missingness Dependency c
fig_missing.add_vline(x=observed_diff_missing, line_dash="dash",
                      line_color="red", annotation_text="Observed Abs Diff")
fig_missing.show()

# Print results
print(f"Observed Absolute Difference in n_ingredients: {observed_diff_missir
print(f"p-value: {p_value_missing:.4f}")

# Interpretation
if p_value_missing < 0.01:
    print("Reject H₀: Missingness of rating is dependent on n_ingredients")
else:
    print("Fail to reject H₀: No strong evidence that missingness of rating

# Optionally, save the plot
fig_missing.write_html('n_ingredients_missing.html', include_plotlyjs='cdn')
```

Loading [MathJax]/extensions/Safe.js




```
Observed Absolute Difference in n_ingredients: 0.1607
p-value: 0.0000
Reject H₀: Missingness of rating is dependent on n_ingredients
```

Our **Observed Absolute Difference in N_ingredients** is **0.1607**. We then get **p-value** of **0.0**, which is less than **0.01**. As a result, we **reject** the null hypothesis and conclude that the missingness of `rating` **does** depend on the number of ingredients(`n_ingredients`)

Moving on, we would also like to see whether the duration of cooking has an impact on the missingness of `rating`.

`minutes` and `rating`'s missing dependecy

Null Hypothesis: The missingness of ratings does not depend on the duration of cooking the recipe.

Alternative Hypothesis: The missingness of ratings does depend on the duration of cookinghe recipe.

Test Statistic: The absolute difference in mean `minutes` between missing and non-missing rating groups.

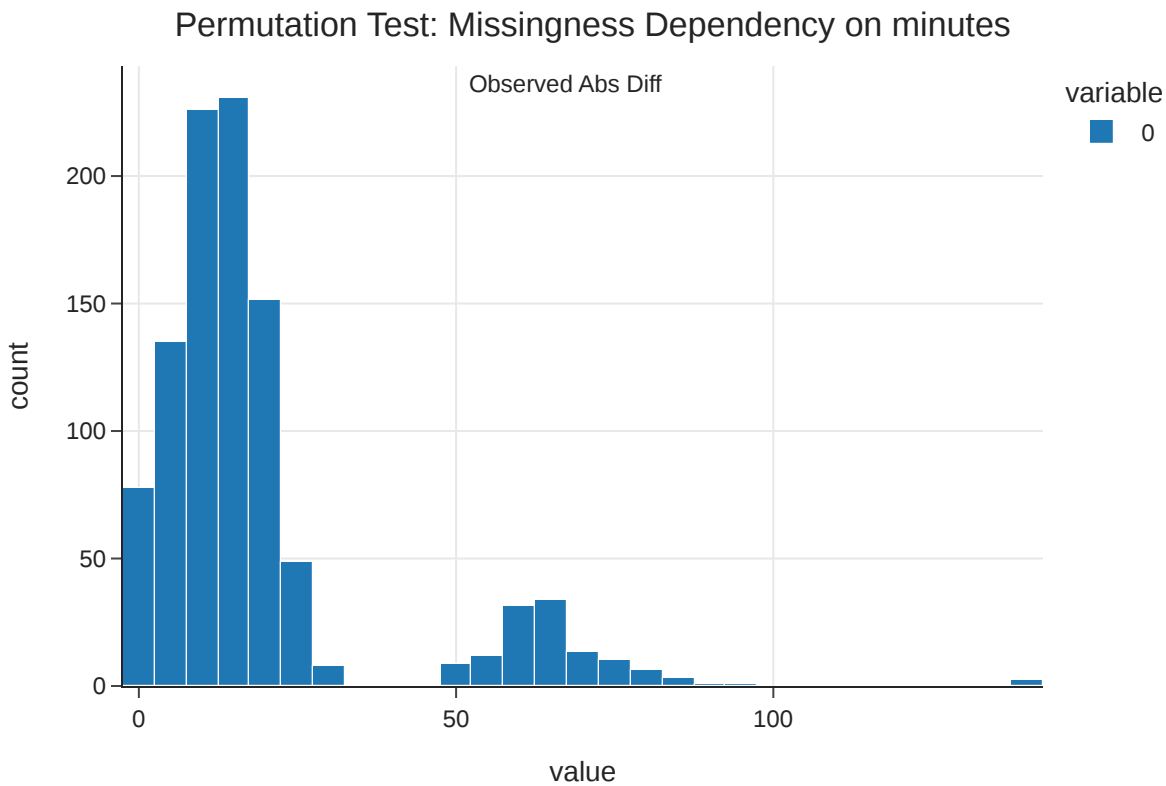
Significance level: 0.01

In [13]:
```
import numpy as np
import pandas as pd
import plotly.express as px
y.stats import norm
```

Loading [MathJax]/extensions/Safe.js

```
df = cleaned_df.copy()
df = df.reset_index()[['name', 'minutes', 'rating']]

# Create missingness indicator for rating
df["rating_missing"] = df["rating"].isna()

# Compute observed absolute difference in mean n_ingredients between missing
observed_diff_missing = abs(df.groupby("rating_missing")["minutes"].mean().c

# Permutation test setup
num_permutations = 1000
perm_diffs_missing = []

for _ in range(num_permutations):
    permuted = df.copy()
    permuted["rating_missing"] = np.random.permutation(permuted["rating_miss
    perm_diff = abs(permuted.groupby("rating_missing")["minutes"].mean().dif
    perm_diffs_missing.append(perm_diff)

# Compute p-value (two-sided)
p_value_missing = np.mean(np.array(perm_diffs_missing) >= observed_diff_miss

# Plot permutation results
fig_missing = px.histogram(perm_diffs_missing, nbins=30,
                           title="Permutation Test: Missingness Dependency c
fig_missing.add_vline(x=observed_diff_missing, line_dash="dash",
                      line_color="red", annotation_text="Observed Abs Diff")
fig_missing.show()

# Print results
print(f"Observed Absolute Difference in minutes: {observed_diff_missing:.4f}
print(f"p-value: {p_value_missing:.4f}")

# Interpretation
if p_value_missing < 0.01:
    print("Reject H₀: Missingness of rating is dependent on minutes")
else:
    print("Fail to reject H₀: No strong evidence that missingness of rating

# Optionally, save the plot
fig_missing.write_html('minutes_missing.html', include_plotlyjs='cdn')
```

Loading [MathJax]/extensions/Safe.js




```
Observed Absolute Difference in minutes: 51.4524
p-value: 0.1170
Fail to reject H₀: No strong evidence that missingness of rating depends on
minutes
```

Our **Observed Absolute Difference in Minutes** is **51.4524**. We then get **p-value** of **0.1050**, which is greater than **0.01**. As a result, we **failed to reject** the null hypothesis and conclude that the missingness of `rating` **there is lack of evidence proving** that the missingness depends on the cooking time of the recipe

In [ ]:

```

```

Step 4: Hypothesis Testing

From the beginning, we would like to predict the rating of a recipe and we suspect that having `meat` in the recipes' `tags` have lower `rating` on average thnn those do not have.

To investigate this question, we designed a **permutaiton test** where we shuffled the boolean column we created `contains_meat`. Our hypothesis, decision rule, statistics, prodcedures are as follows:

Null Hypothesis: People rate meat and non-meat tagged dishes the same

Alternative Hypothesis: People rate recipes with **meat** tagged lower than they rate non-meat tagged ones

Loading [MathJax]/extensions/Safe.js

Decision Rule: We will reject the null hypothesis if our p-value is less than **significance level** 0.05

Test Statistic: We used **difference in mean** between ratings of non-meat dishes and meat dishes(with meat-without meat) as test stats since it is a directional test

Prodcedures: We run a 10000-simulation permutaiton test in order to get the empirical distribution of the test statistics under the null hypothesis. Since we lack prior information about the underlying population, so instead of relying on assumptions about the data, the permutation test allows us to directly assess whether the two observed distributions (recipes with meat and without meat) are likely to have come from the same population. We randomly shuffled the bool values given by `contains_meat` column many times and recalculating the rating differences each time, then we build an empirical distribution of the difference under the null hypothesis (that meat inclusion does not affect ratings).

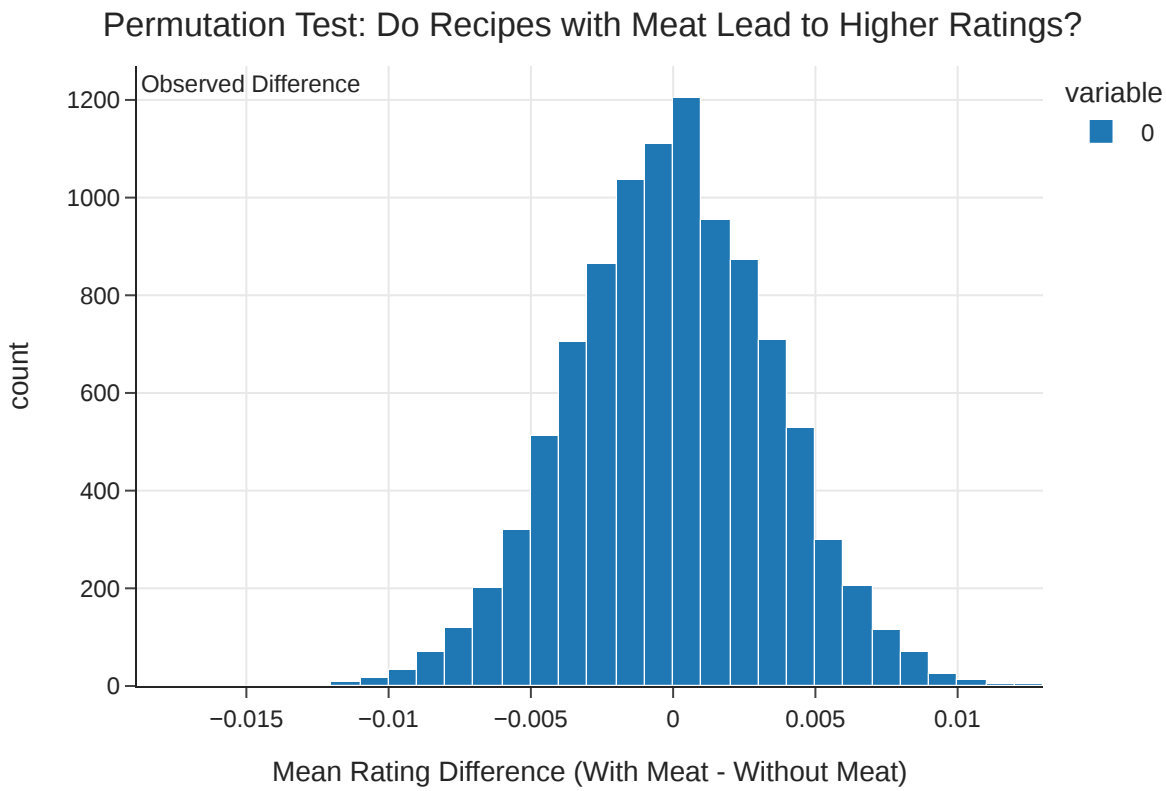
Here is our **result**:

In [14]:
```
import numpy as np
import pandas as pd
import plotly.express as px


df = cleaned_df.copy()
df = df.reset_index()[['name', 'contains_meat', 'rating']]


df_clean = df.dropna(subset=["rating"])

# Ensure that 'contains_meat' is boolean
df_clean["contains_meat"] = df_clean["contains_meat"].astype(bool)


observed_diff_rating = (
    df_clean.groupby("contains_meat")["rating"].mean().loc[True]
    - df_clean.groupby("contains_meat")["rating"].mean().loc[False]
)

num_permutations = 10000
perm_diffs_rating = []

# Set seed for reproducibility
np.random.seed(42)

for _ in range(num_permutations):
    permuted = df_clean.copy()
    # Permute the contains_meat labels
    permuted["contains_meat"] = np.random.permutation(permuted["contains_mea
    perm_mean_diff = (
        permuted.groupby("contains_meat")["rating"].mean().loc[True]
```

Loading [MathJax]/extensions/Safe.js

```
        - permuted.groupby("contains_meat")["rating"].mean().loc[False]
    )
    perm_diffs_rating.append(perm_mean_diff)


p_value_rating = np.mean(np.array(perm_diffs_rating) <= observed_diff_rating


fig_rating = px.histogram(
    perm_diffs_rating,
    nbins=30,
    title="Permutation Test: Do Recipes with Meat Lead to Higher Ratings?",
    labels={"value": "Mean Rating Difference (With Meat - Without Meat)"}
)
fig_rating.add_vline(
    x=observed_diff_rating,
    line_dash="dash",
    line_color="red",
    annotation_text="Observed Difference",
)
fig_rating.show()

# Print results
print(f"Observed Difference in Rating (With Meat - Without Meat): {observed_
print(f"p-value: {p_value_rating:.4f}")

# Interpretation
if p_value_rating < 0.05:
    print("Reject H₀: There is evidence that recipes with meat lead to lower
else:
    print("Fail to reject H₀: No strong evidence that meat inclusion affects


fig_rating.write_html('permutation.html', include_plotlyjs='cdn')
```

Loading [MathJax]/extensions/Safe.js




```
Observed Difference in Rating (With Meat - Without Meat): -0.0188
p-value: 0.0000
Reject H₀: There is evidence that recipes with meat lead to lower ratings.
```

Our **Observed Difference in Rating** (With Meat - Without Meat) is **-0.0188**. We then get **p-value** of **0.0**, which is less than **0.05**. As a result, we **reject** the null hypothesis and conclude that people rate recipes with meat in their tag lower than recipes without meat in their tag.

Step 5: Framing a Prediction Problem

In this project, we will be **Predicting the Rating of a Recipe**, which is a **multi-class classification problem**, as `rating` consists only the integer from 1-5 (note we replace the **0** with **np.nan** at the beginning), which would be an catagorical, ordinal variable.

We chose `rating` as our response variable as it is the statistic that best represent user satisfaction and overall quality. Additionally, we conduct several visualizations and analysis with `rating` as one of the variables and observed several trends across the distributions of `rating`. For instance, our hypothesis shows people rate recipes with meat in their tag **lower** than recipes without meat in their tag. So we may be able to predict the rating through features like inclusion of meat.

At the time of prediction, some avaliable features include `contains_meat` and `minutes` since they were all avalible as soon as the recipes were created

Loading [MathJax]/extensions/Safe.js

before the actual ratings were given, we can safely use them as suitable predictors for our model.

To evaluate our model's effectiveness, we will take a look at both the model's **accuracy** and **F1-score**.

Accuracy: The accuracy will allow us to analyze overall how our model is doing in terms of creating predictions based on **given** parameters, in this case being `minute` and `contains_meat`

F1-Score: The f1 socre will calculate harmonic mean of precision and recall for each class, weighted by the class frequency. As shown in the above analysis, we observed the imbalanced distribution for `rating`, which are heavily skewed left with most around 4 to 5. So, using such metric calculation ensures that the performance on less frequent rating categories is properly accounted for

Step 6: Baseline Model

In [15]:
```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.compose import make_column_transformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import root_mean_squared_error
from sklearn.metrics import accuracy_score, f1_score
```

We used the **Random Forest Classifier** with two parameters(one **quantitative** one **nominal**) for the baseline model.

1. `contains_meat`: As in the hypothesis section mentioned, people are likely to rate dishes with meat lower than the dishes without meat. This is a **nominal** variable contains **bool** value. We transformed the booleans into integers using **OneHotEncoder()** with 1 means True and 0 means False.

2. `protein` : The `protein` column contains continuous, numerical values with **float** type, making it **quantitative**. Speicifcally, it represents the **percentage of daily value** of **protein** for the speicifc recipe. We used **StandardScaler()** to standardize the data.

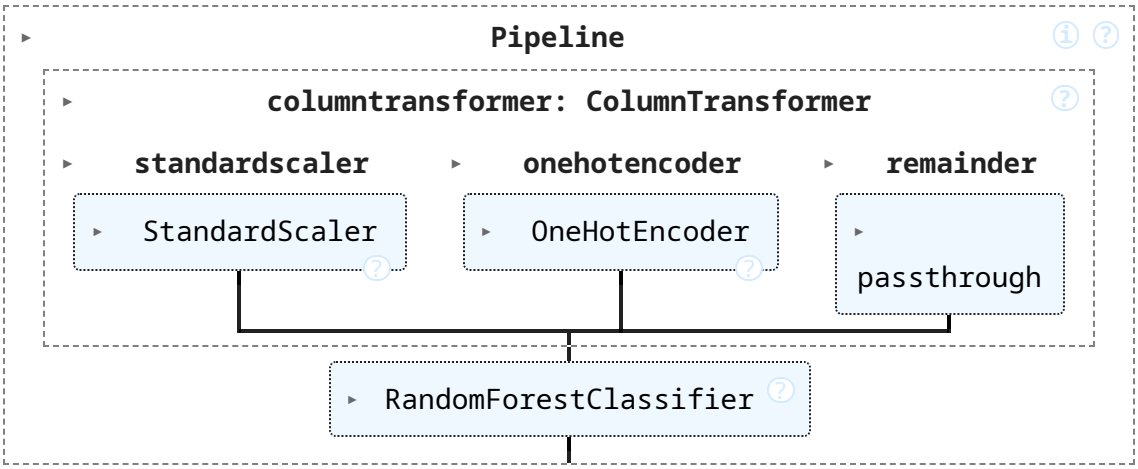
Additionally, we seperate the data into training group and testing group. The test group contains 20% of the data while the train group contians 80% of the data.

In [16]:
```
df=cleaned_df.dropna(subset='rating')

X= df[['protein','contains_meat']]
```

Loading [MathJax]/extensions/Safe.js

```
y=df['rating']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,ranc

preproc = make_column_transformer(
    (StandardScaler(),['protein']),
    (OneHotEncoder(drop = 'first'), ['contains_meat']),
    remainder='passthrough',
)
pl = make_pipeline(preproc, RandomForestClassifier(  # Uses 100 separate dec
    random_state=42))

pl.fit(X_train,y_train)
```

Out[16]:



In [17]:

```
y_pred = pl.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average='macro')

print('Accuracy is:', accuracy)
print("f1 score is:", f1)
```

```
Accuracy is: 0.7736046856127077
f1 score is: 0.17489827929656016
```

In [18]:

```
# Extract the RandomForestClassifier from the pipeline.
rf = pl.named_steps['randomforestclassifier']

# Get the raw feature importances from the classifier.
importances = rf.feature_importances_

# Get the column transformer from the pipeline.
col_transformer = pl.named_steps['columntransformer']

# If your transformers support get_feature_names_out, you can do:
feature_names = col_transformer.get_feature_names_out()

# Now, you can pair each feature name with its importance:
for name, importance in zip(feature_names, importances):
    print(f"{name}: {importance:.4f}")
```

```
standardscaler__protein: 0.9789
der__contains_meat_True: 0.0211
```

Loading [MathJax]/extensions/Safe.js

The statistics produced by the baseline model is as follows:

Accuracy 0.7736046856127077

F1-Score 0.17489827929656016

Our baseline has a pretty high accuracy but low F1-score. The f1-score shows that our baseline model is not overconfident and does not miss to many actual positives. In other words while the model is often correct, it likely performs very poorly on one or more classes—usually the minority class—resulting in low precision, recall, or both when these are combined as the F1 score. This is most likely due to the **imbalanced** distribution, as mentioned above during our analysis in EDA.

To improve our model, we decided to take a look into the feature importance:

For two of our **raw features: protein, contains_meat_True**, we have correponding transformed feature importance:

standardscaler__protein 0.9789

onehotencoder__contains_meat_True 0.0211 This shows that the model relies heavily on the protein feature to make predictions, while the indicator for whether a recipe contains meat plays a much smaller role.

Step 7: Final Model

In order to build a final model, we decided to do another round of exploratory analysis of all the **numerical** columns. Speicifcally, we would like to examine and take a look at the **distribution** and **correlation** with target column `rating` of each.

First, we started off with create a temporary dataframe **df** for simplicity. It is a copy of our **cleaned_df** with all rows containing at least one **np.nan** being removed.

To make full use of our **datetime** columns, we split the **submitted** columns based on `day`, `month`, and `year`, and created a column for each, and we convert `date` columns to **ordinal** numbers as well.

In [19]:
```
df = cleaned_df.copy().reset_index().dropna()
df['day'] = df['submitted'].dt.day
df['month'] = df['submitted'].dt.month
df['year'] =df['submitted'].dt.year
df['date_ordinal'] = df['date'].apply(lambda x: x.toordinal())
```

Loading [MathJax]/extensions/Safe.js

We then created another dataframe **df_numeric**, extracting all the relevant **numerical** columns from **df** we would like to examine and stored the names in a varibel **relevant_columns**, containing `minutes`, `calories`, `total_fat`, `sugar`, `sodium`, `protein`, `saturated_fat`, `carbohydrates`, `n_steps`, `n_ingredients`, `submitted`, `date_ordinal`, `day`, `year`, `month`.

Looping through each column, we create graphs to visualize each **distribution**.

In [27]:
```
df_numeric = df.select_dtypes(include=[np.number])
relevant_columns = ['minutes', 'n_steps', 'n_ingredients','calories',
       'total_fat', 'sugar', 'sodium', 'protein', 'saturated_fat',
       'carbohydrates', 'day', 'month', 'year','date_ordinal','rating']
df_numeric = df_numeric[relevant_columns]
```

In [ ]:
```
for col in relevant_columns:
    plt.figure(figsize=(8, 4))
    sns.histplot(df[col], kde=True)
    plt.title(f"Distribution of {col}")
    plt.show()
```

We made some key observations after the visualizations:

1. Some distributions like `n_steps` and `n_ingredients` look fairly symmetric, indicating that the mean and median are likely similar. For these we can just apply some simple **StandardScaler()**.

2. Other distributions especially for those relevant to **nutrition**, such as `protein` and `saturated_fat`, and `minutes` show clear skewness (either right-skewed or left-skewed), which implies that a few extreme values are pulling the mean away from the median. For these columns, we might need to apply potential transoformers and scalers like **log or exponential transformation** or **RobustScaler()**.

3. Most of the **datetime** type columns like `day` and `month` appeared to be relatively even, uniform, and symmetric.

In general, our data seems to be quite **imbalanced**. Due to this consideration, for **model selection**, we decided to keep using **Random Forest Classifiers**, which is tend to be robust to outliers and skewed distributions because they split data based on thresholds and are less affected by extreme values.

Next, we decided to take more insights by looking into the **linear correlation** of each with our targeted/predicted column `rating`.

In [28]:
```
correlation_scores = df_numeric.corr()['rating'].sort_values(ascending=False
correlation_scores
```

Loading [MathJax]/extensions/Safe.js

Out[28]:
```
rating           1.00e+00
year             1.89e-02
month            5.06e-03
                   ...
protein         -1.20e-02
carbohydrates   -1.29e-02
date_ordinal    -5.51e-02
Name: rating, Length: 15, dtype: float64
```

Many variables and **nutritional metrics** (total_fat, saturated_fat, sodium, sugar, calories, protein, carbohydrates) have correlations near zero (ranging from about **0.02463** to **-0.012863**), indicates that these factors have **little** linear association with the rating in this dataset.

This further proves that in order to increase predictive power, we have to create more informative features by using techniques such as **creating interaction terms** and **apply transformations** that better capture the underlying relationships.

1. Numerical(Nutritional) Features:

`calories`, `protein`, `total fat`, `sugar`, `sodium`, `saturated fat`, and `carbohydrates`

According to the pivot table, we know that nutritional information generally are good indicator for `rating`.(As `rating` increases, in general, all nutritional features decrease) To address skewness and redundancy as shown in the above analysis, a custom log transformation (np.log1p) is applied to nutrition features, and then we utilized **RobustScaler()** to scale them considering the high potential of outliers.

In [29]:
```
# ================================
# Define log-transform for nutrition features
# ================================
def log_transform_nutrition(X):
    X = X.copy()
    nutrition_cols = ['calories', 'protein', 'total_fat', 'sugar', 'sodium',
    for col in nutrition_cols:
        # Use np.log1p to handle zeros gracefully
        X[col] = np.log1p(X[col])
    return X

# ================================
# Nutrition pipeline: log-transform, scale, then apply PCA
# ================================
nutrition_pipeline = SkPipeline([
    ('log_transform', FunctionTransformer(log_transform_nutrition, validate=
    ('scaler', RobustScaler())
])
```

Loading [MathJax]/extensions/Safe.js

2. Preparation Details:

`minutes`, `n_ingredients` and `n_steps`

Features that are normally distributed like number of steps, and ingredients are scaled using **StandardScaler()**. We also know from previous analysis that higher rated recipes are more time-consuming and require more comlex procedures.

In [30]:
```
# ================================
# Pipeline for other numeric features (e.g., minutes, n_steps, n_ingredients
# ================================
other_numeric_columns = ['minutes', 'n_steps', 'n_ingredients']
other_numeric_pipeline = SkPipeline([
    ('scaler', StandardScaler())
])
```

3. Categorical Features:

`contains_meat`

We keep using the **OneHotEncode()** as shown in our baseline model as it helps with achieving decent accuracy

In [31]:
```
# ================================
# Pipeline for categorical feature: contains_meat
# ================================
categorical_pipeline = SkPipeline([
    ('onehot', OneHotEncoder(drop='first'))
])
```

4. Textual Features:

`tags`

Since it contians a list of tags for a given recipe, we preprocessed them by cleaning, lowercasing, and concatenating individual tags. They are then transformed into numerical features using TF-IDF. We think that there might be certain tags that are associated with higher views, search, and attentions, such as those related to holidays or special diet plan.

In [32]:
```
# ================================
# TF-IDF Transformer for Tags
# ================================
def join_tags(x):
    if isinstance(x, list):
        cleaned = [str(item).replace("'", "").replace('"', "").strip().lower
        joined = " ".join(cleaned)
```

Loading [MathJax]/extensions/Safe.js

```
    else:
        joined = str(x).strip().lower()
    return joined if joined else "none"

class TfidfTagsTransformer(BaseEstimator, TransformerMixin):
    def __init__(self, min_df=1, stop_words=None, **kwargs):
        self.min_df = min_df
        self.stop_words = stop_words
        self.kwargs = kwargs
        self.vectorizer = TfidfVectorizer(min_df=self.min_df,
                                          stop_words=self.stop_words,
                                          **self.kwargs)

    def fit(self, X, y=None):
        if isinstance(X, pd.DataFrame):
            X = X.squeeze()
        X_text = [join_tags(x) for x in X]
        self.vectorizer.fit(X_text)
        return self

    def transform(self, X, y=None):
        if isinstance(X, pd.DataFrame):
            X = X.squeeze()
        X_text = [join_tags(x) for x in X]
        return self.vectorizer.transform(X_text)

    def get_feature_names_out(self, input_features=None):
        return self.vectorizer.get_feature_names_out()

tags_pipeline = SkPipeline([
    ('tfidf_tags', TfidfTagsTransformer(min_df=1, stop_words='english'))
])
```

6. Date Features:

`submitted` and `date`

Since these were columns containing datetime objects, we think that recipes rating might vary in certain holidays, seasons, culture, societal trend, and other timeframe. Also, we found that higher ratings are given in earlier submitted years. To account for these, we further extract day, month, year, day of week, and ordinal values from the submitted and recipe dates to capture temporal patterns. We then applied **StandardScaler()** to scale them.

In [33]:
```
# ================================
# Date Pipeline: extract date features and scale them
# ================================
def extract_date_features(X):
    X = X.copy()
    # Parse dates (assuming valid formats since there are no missing values)
    X['submitted'] = pd.to_datetime(X['submitted'], errors='coerce')
    X['date'] = pd.to_datetime(X['date'], errors='coerce')
```

Loading [MathJax]/extensions/Safe.js

```
    # Extract basic date features
    X['day'] = X['submitted'].dt.day
    X['month'] = X['submitted'].dt.month
    X['year'] = X['submitted'].dt.year
    X['day_of_week']=X['submitted'].dt.dayofweek

    # Convert dates to ordinal values
    X['submitted_ordinal'] = X['submitted'].apply(lambda x: x.toordinal())
    X['date_ordinal'] = X['date'].apply(lambda x: x.toordinal())

    return X[['day', 'month', 'year', 'day_of_week','submitted_ordinal', 'da

date_pipeline = Pipeline([
    ('date_features', FunctionTransformer(extract_date_features, validate=Fa
    ('scaler', StandardScaler())
])
```

7. Asemble All the pipelines into final pipeline with Classifier

In [34]:

```
# ================================
# Combine all pipelines using ColumnTransformer
# ================================
preprocessor = ColumnTransformer(transformers=[
    ('nutrition', nutrition_pipeline, ['calories', 'protein', 'total_fat', '
    ('other_num', other_numeric_pipeline, other_numeric_columns),
    ('cat', categorical_pipeline, ['contains_meat']),
    ('tags', tags_pipeline, ['tags']),
    ('dates', date_pipeline, ['submitted', 'date'])
], remainder='drop')

# ================================
# Final Pipeline with Classifier
# ================================
final_pipeline = SkPipeline([
    ('preprocessor', preprocessor),
    ('clf', RandomForestClassifier(random_state=42))
])
```

9. Split data and Set up grid search, train the model(since the whole model took hours to finish one round, we only recorded the data on website:)

In [43]:

```

```

In [ ]:

```
X = df[['calories', 'protein', 'total_fat', 'sugar', 'date', 'tags', 'sodium
        'carbohydrates', 'minutes', 'n_steps', 'n_ingredients', 'contains_me
y = df['rating']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rar

kf = KFold(n_splits=5, shuffle=True, random_state=42)
param_grid = {
    'clf__max_depth': [3, 5, 7, None],
```

Loading [MathJax]/extensions/Safe.js

```
    'clf__n_estimators': [100, 200, 300, 500],
    'clf__min_samples_split': [2, 5, 10],
    'clf__min_samples_leaf': [1, 2, 4],
    'clf__max_features': ['sqrt', 'log2', None]
}

grid_search = GridSearchCV(final_pipeline, param_grid, cv=kf, scoring='f1_ma
grid_search.fit(X_train, y_train)

print("Best Hyperparameters:", grid_search.best_params_)
print("Best Cross-Validated f1_macro:", grid_search.best_score_)

final_model = grid_search.best_estimator_
test_accuracy = final_model.score(X_test, y_test)
print("Test Set Accuracy:", test_accuracy)
```

In [ ]:

```
```

Step 8: Fairness Analysis

For our fairness analysis, we decided to use the column `contains_meat` to create the group. **Group X :** Defined as all recipes where the boolean column `contains_meat` is True. **Group Y :** Defined as all recipes where the boolean column `contains_meat` is False. We used **precision/recall parity** as our evaluation metric, as seen before. The metric calculates the precision for each class and then takes the average, ensuring that performance across all classes is considered equally. In imbalanced datasets or multi-class settings, overall accuracy can be misleading. It's more important to focus on **False Positives**, that is, for the model to correctly identify the rating of a recipe among all instances of that rating. False positives would not be good since it would mislead users with the incorrectly labeled ratings.

Null Hypothesis: The model is fair. That is, the difference in macro precision between recipes with meat (Group X) and recipes without meat (Group Y) is zero, and any observed difference is due to random chance.

Alternative Hypothesis: The model is unfair. In other words, there is a systematic difference in macro precision between the two groups (specifically, that recipes without meat have a higher precision than those with meat).

Decision Rule: We will reject the null hypothesis if our p-value is less than **significance level** 0.05

Test Statistic: We used **difference in macro precision** between between the two groups(without meat-with meat) as our test statistic.

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
```

Loading [MathJax]/extensions/Safe.js

```
from sklearn.metrics import precision_score

# Get predictions from the final model on the test set.
y_pred = final_model.predict(X_test)

group_labels = X_test['contains_meat'].values


mask_meat = group_labels == True    # Group X: recipes with meat
mask_no_meat = group_labels == False  # Group Y: recipes without meat

precision_meat = precision_score(y_test[mask_meat], y_pred[mask_meat], avera
precision_no_meat = precision_score(y_test[mask_no_meat], y_pred[mask_no_mea


observed_diff = precision_no_meat - precision_meat

print("Observed Precision:")
print("Recipes WITHOUT meat (contains_meat == False):", precision_no_meat)
print("Recipes WITH meat (contains_meat == True):", precision_meat)
print("Observed Difference (non-meat minus meat):", observed_diff)

num_permutations = 1000
permuted_diffs = np.zeros(num_permutations)

for i in range(num_permutations):

    permuted_labels = np.random.permutation(group_labels)

    perm_mask_meat = permuted_labels == True
    perm_mask_no_meat = permuted_labels == False

    perm_precision_meat = precision_score(y_test[perm_mask_meat], y_pred[per
    perm_precision_no_meat = precision_score(y_test[perm_mask_no_meat], y_pr

    permuted_diffs[i] = perm_precision_no_meat - perm_precision_meat


p_value = np.mean(permuted_diffs >= observed_diff)
print("Permutation Test p-value:", p_value)

alpha = 0.01  # significance level
if p_value < alpha:
    print(f"At the significance level of {alpha}, we reject the null hypothe
else:
    print(f"At the significance level of {alpha}, we fail to reject the null

plt.figure(figsize=(8, 6))
plt.hist(permuted_diffs, bins=30, alpha=0.75, color='skyblue', edgecolor='bl
plt.axvline(observed_diff, color='red', linestyle='dashed', linewidth=2,
            label=f'Observed Diff = {observed_diff:.4f}')
plt.xlabel("Difference in Macro Precision (Non-meat minus Meat)")
plt.ylabel("Frequency")
plt.title("Permutation Test: Precision Difference Distribution")
plt.legend()
plt.show()
```

Loading [MathJax]/extensions/Safe.js

Our **Observed Difference in scores** is **0.13079558064360503**. We then get **p-value** of **0.071**, which is less than **0.01**. As a result, we **fail to reject** the null hypothesis. This suggests that the observed difference in precision could reasonably be due to random chance, and we conclude that there is no statistically significant evidence proving that our model is unfair.

In [ ]:

```

```

In [ ]:

```

```

This notebook was converted with convert.ploomber.io

Loading [MathJax]/extensions/Safe.js